Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №4
По дисциплине «Интеллектуальный анализ данных»
Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Сугак В.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM.

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.

2. Сравнить результаты, полученные при

- обучении без предобучения (ЛР 3);
- обучении с предобучением, используя автоэнкодерный подход (ЛР3);
- обучении с предобучением, используя RBM.

3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;

4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№	Выборка	Тип задачи	Целевая переменная
17	https://archive.ics.uci.edu/dataset/503/hepatitis+c+virus+hcv+for+egyptian+patients	классификация	Baselinehistological staging

Ход работы

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from ucimlrepo import fetch_ucirepo

HIDDEN = [64, 32, 16]
AE_EPOCHS = 30
RBM_EPOCHS = 30
FT_EPOCHS = 60
BATCH = 64
LR_AE = 1e-3
LR_FT = 1e-3
LR_RBM = 0.01
LR_GRBM = 0.002
SEEDS = [0, 1, 2, 3, 4]
MODES = ["без предобучения", "автоэнкодер", "RBM"]
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
def load_hcv():
    ds = fetch_ucirepo(id=503)
    X = ds.data.features.copy()
    y = ds.data.targets.iloc[:, 0]
    X = X.apply(pd.to_numeric, errors="coerce").fillna(X.median(numeric_only=True))
    return X.values.astype(np.float32), y.values, "numeric"
```

```
def load_mushroom():
    ds = fetch_ucirepo(id=73)
    df = pd.concat([ds.data.features, ds.data.targets], axis=1)
    df = df.replace("?", np.nan).dropna()
    y = df["poisonous"].values
    X = df.drop(columns=["poisonous"])
    return X, y, "categorical"
```

```
def prepare(X, y, kind, seed):
    y = LabelEncoder().fit_transform(y)
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.3, stratify=y, random_state=seed)
    if kind == "numeric":
        sc = StandardScaler().fit(X_tr)
        X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)
    else:
        enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(X_tr)
        X_tr, X_te = enc.transform(X_tr), enc.transform(X_te)
    return (X_tr.astype(np.float32), X_te.astype(np.float32),
            y_tr.astype(np.int64), y_te.astype(np.int64))
```

```
def build_mlp(in_dim, n_classes):
    layers, prev = [], in_dim
    for h in HIDDEN:
        layers += [nn.Linear(prev, h), nn.ReLU()]
        prev = h
    layers.append(nn.Linear(prev, n_classes))
    return nn.Sequential(*layers)
```

```
    def train_classifier(model, X_tr, y_tr, X_te, y_te, seed):
        torch.manual_seed(seed)
        model.to(device)
        opt = optim.Adam(model.parameters(), lr=LR_FT)
        crit = nn.CrossEntropyLoss()
        loader = DataLoader(TensorDataset(torch.tensor(X_tr), torch.tensor(y_tr)),
                            batch_size=BATCH, shuffle=True)
        Xte, yte = torch.tensor(X_te).to(device), torch.tensor(y_te).to(device)
        hist = {"train_loss": [], "test_loss": []}
        for _ in range(FT_EPOCHS):
            model.train()
            tot = 0.0
            for xb, yb in loader:
                xb, yb = xb.to(device), yb.to(device)
                opt.zero_grad()
                loss = crit(model(xb), yb)
                loss.backward()
                opt.step()
                tot += loss.item() * len(xb)
            hist["train_loss"].append(tot / len(X_tr))
            model.eval()
            with torch.no_grad():
                hist["test_loss"].append(crit(model(Xte), yte).item())
        model.eval()
        with torch.no_grad():
            pred = model(Xte).argmax(1).cpu().numpy()
        return pred, hist

    def linear_layers_of(model):
        return [m for m in model if isinstance(m, nn.Linear)][:-1]


    def pretrain_autoencoder(model, X_tr, seed):
        torch.manual_seed(seed)
        H = torch.tensor(X_tr).to(device)
        for layer in linear_layers_of(model):
            enc = nn.Sequential(nn.Linear(layer.in_features, layer.out_features), nn.ReLU()).to(device)
            dec = nn.Linear(layer.out_features, layer.in_features).to(device)
            opt = optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=LR_AE)
            crit = nn.MSELoss()
            loader = DataLoader(TensorDataset(H), batch_size=BATCH, shuffle=True)
            for _ in range(AE_EPOCHS):
                for (xb,) in loader:
                    opt.zero_grad()
                    crit(dec(enc(xb)), xb).backward()
                    opt.step()
            layer.weight.data.copy_(enc[0].weight.data)
            layer.bias.data.copy_(enc[0].bias.data)
            with torch.no_grad():
                H = enc(H)


    class RBM:
        def __init__(self, n_vis, n_hid, gaussian_visible=False, lr=0.01):
            self.W = (0.01 * torch.randn(n_vis, n_hid)).to(device)
            self.b = torch.zeros(n_vis, device=device)
            self.c = torch.zeros(n_hid, device=device)
            self.gaussian = gaussian_visible
            self.lr = lr

        def hidden_prob(self, v):
            return torch.sigmoid(v @ self.W + self.c)

        def visible_mean(self, h):
            a = h @ self.W.T + self.b
            return a if self.gaussian else torch.sigmoid(a)

        def fit(self, V, epochs, batch=BATCH):
            loader = DataLoader(TensorDataset(V), batch_size=batch, shuffle=True)
            errors = []
            for _ in range(epochs):
                tot = 0.0
                for (v0,) in loader:
                    ph0 = self.hidden_prob(v0)
                    h0 = torch.bernoulli(ph0)
                    v1 = self.visible_mean(h0)
                    ph1 = self.hidden_prob(v1)
                    n = v0.shape[0]
                    self.W += self.lr * (v0.T @ ph0 - v1.T @ ph1) / n
                    self.b += self.lr * (v0 - v1).mean(0)
                    self.c += self.lr * (ph0 - ph1).mean(0)
                    tot += ((v0 - v1) ** 2).mean().item() * n
                errors.append(tot / len(V))
            return errors


    def _is_binary(X):
        return bool(np.isin(X, [0.0, 1.0]).all())


    def pretrain_rbm(model, X_tr, seed):
        torch.manual_seed(seed)
        V = torch.tensor(X_tr).to(device)
        errs_all = []
        for k, layer in enumerate(linear_layers_of(model)):
            gauss = (k == 0) and not _is_binary(X_tr)
            rbm = RBM(layer.in_features, layer.out_features,
                      gaussian_visible=gauss, lr=LR_GRBM if gauss else LR_RBM)
            errs_all.append(rbm.fit(V, RBM_EPOCHS))
            layer.weight.data.copy_(rbm.W.T)
            layer.bias.data.copy_(rbm.c)
            with torch.no_grad():
                V = rbm.hidden_prob(V)
        return errs_all


    def run_dataset(name, X, y, kind):
        print(f"\n{'=' * 20} {name} {'=' * 20}")
        rows, last = [], {}
        for seed in SEEDS:
```

```
            X_tr, X_te, y_tr, y_te = prepare(X, y,
kind, seed)
            n_classes = len(np.unique(y_tr))
            for mode in MODES:
                torch.manual_seed(seed)
                model = build_mlp(X_tr.shape[1],
n_classes)
                pre = None
                if mode == "автоэнкодер":
                    pretrain_autoencoder(model, X_tr,
seed)
                elif mode == "RBM":
                    pre = pretrain_rbm(model, X_tr,
seed)
                pred, hist = train_classifier(model,
X_tr, y_tr, X_te, y_te, seed)
                rows.append({"seed": seed, "mode":
mode,
                             "accuracy":
accuracy_score(y_te, pred),
                             "f1_macro":
f1_score(y_te, pred, average="macro"),
                             "f1_weighted":
f1_score(y_te, pred, average="weighted")})
                last[mode] = dict(y_te=y_te,
pred=pred, hist=hist, pre=pre)

        res = pd.DataFrame(rows)
        res.to_csv(f"lab4_results_{name}.csv",
index=False)
        summary = res.groupby("mode")[["accuracy",
"f1_macro", "f1_weighted"]].agg(["mean", "std"])

print(summary.reindex(MODES).round(4).to_string())

        fig, ax = plt.subplots(1, 3, figsize=(15,
4.5))
        for i, mode in enumerate(MODES):
            cm = confusion_matrix(last[mode]["y_te"],
last[mode]["pred"])
            ax[i].imshow(cm, cmap="Blues")
            ax[i].set_title(f"{name}: {mode}")
            ax[i].set_xlabel("Предсказано")
            ax[i].set_ylabel("Истина")
            for a in range(cm.shape[0]):
                for b in range(cm.shape[1]):
                    ax[i].text(b, a, cm[a, b],
ha="center", va="center")
        plt.tight_layout()
        plt.savefig(f"lab4_{name}_confusion.png",
dpi=150)
        plt.close()

        fig, ax = plt.subplots(1, 2, figsize=(12,
4.5))
        for mode in MODES:
            ax[0].plot(last[mode]["hist"]["test_loss"], label=mode)
        ax[0].set_title(f"{name}: test loss при
дообучении")
        ax[0].set_xlabel("Эпоха")
        ax[0].legend()
        ax[0].grid(True)
        for k, e in enumerate(last["RBM"]["pre"]):
            ax[1].plot(e, label=f"RBM слой {k + 1}")
        ax[1].set_title(f"{name}: ошибка
реконструкции RBM")
        ax[1].set_xlabel("Эпоха")
        ax[1].set_ylabel("MSE")
        ax[1].legend()
        ax[1].grid(True)
        plt.tight_layout()
        plt.savefig(f"lab4_{name}_curves.png",
dpi=150)
        plt.close()

        means = summary.reindex(MODES)
        plt.figure(figsize=(7, 4.5))
        xs = np.arange(len(MODES))
        plt.bar(xs - 0.2, means[("accuracy", "mean")],
0.4,
                yerr=means[("accuracy", "std")],
label="accuracy")
        plt.bar(xs + 0.2, means[("f1_macro", "mean")],
0.4,
                yerr=means[("f1_macro", "std")],
label="macro F1")
        plt.xticks(xs, MODES)
        plt.title(f"{name}: сравнение режимов (среднее
± std, {len(SEEDS)} запусков)")
        plt.legend()
        plt.tight_layout()
        plt.savefig(f"lab4_{name}_compare.png",
dpi=150)
        plt.close()

        print("\nОтчёт по классам (последний seed,
RBM):")
        print(classification_report(last["RBM"]["y_te"],
        last["RBM"]["pred"], zero_division=0))


if __name__ == "__main__":
    X, y, kind = load_hcv()
    run_dataset("HCV", X, y, kind)
    X, y, kind = load_mushroom()
    run_dataset("Mushroom", X, y, kind)
```

Вывод программы:

```
==================== HCV ====================
                  accuracy          f1_macro          f1_weighted
                      mean     std      mean     std        mean     std
mode
без предобучения    0.2538  0.0208    0.2514  0.0208      0.2526  0.0208
автоэнкодер         0.2577  0.0137    0.2548  0.0132      0.2560  0.0138
RBM                 0.2572  0.0213    0.2489  0.0262      0.2494  0.0257
```

```
Отчёт по классам (последний seed, RBM):
              precision    recall  f1-score   support

           0       0.21      0.37      0.27       101
           1       0.24      0.32      0.27       100
           2       0.26      0.10      0.15       106
           3       0.43      0.25      0.31       109

    accuracy                           0.26       416
   macro avg       0.28      0.26      0.25       416
weighted avg       0.29      0.26      0.25       416

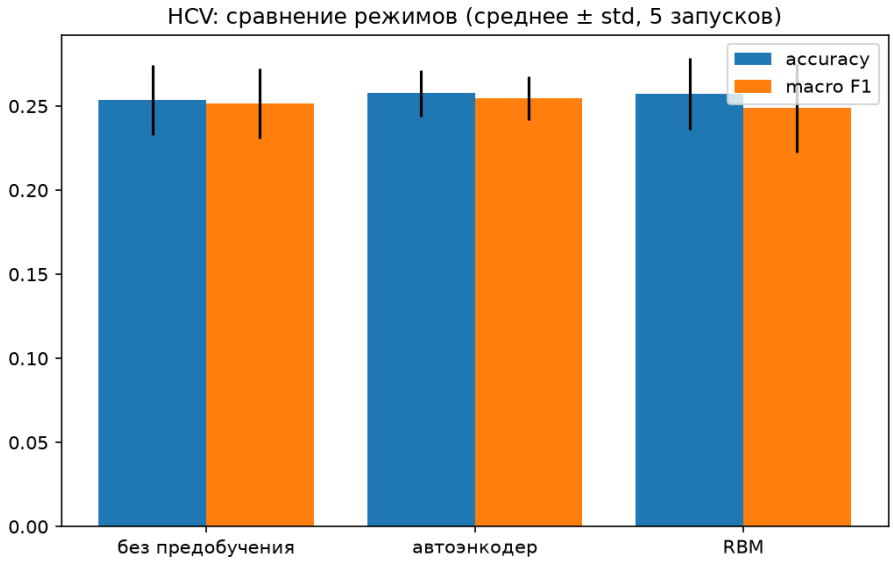

==================== Mushroom ====================
                accuracy      f1_macro      f1_weighted
                    mean  std     mean  std        mean  std
mode
без предобучения     1.0  0.0      1.0  0.0         1.0  0.0
автоэнкодер          1.0  0.0      1.0  0.0         1.0  0.0
RBM                  1.0  0.0      1.0  0.0         1.0  0.0

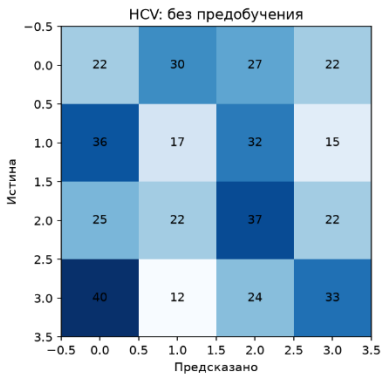
Отчёт по классам (последний seed, RBM):
              precision    recall  f1-score   support

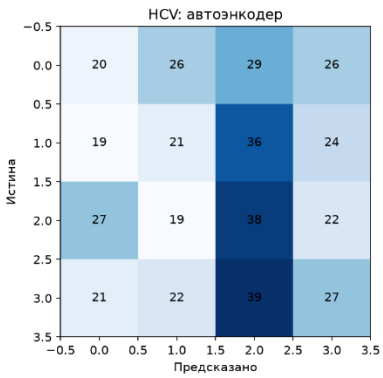
           0       1.00      1.00      1.00      1047
           1       1.00      1.00      1.00       647

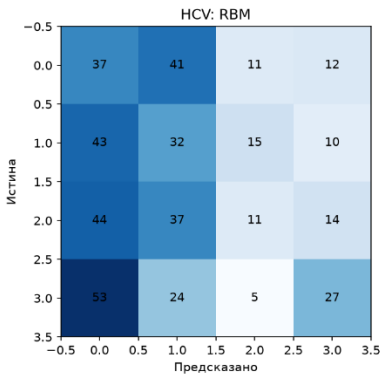
    accuracy                           1.00      1694
   macro avg       1.00      1.00      1.00      1694
weighted avg       1.00      1.00      1.00      1694
```

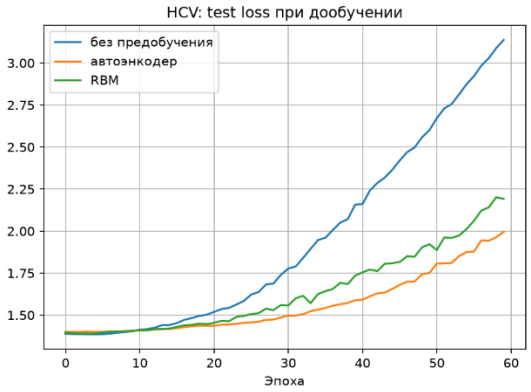
Графики:

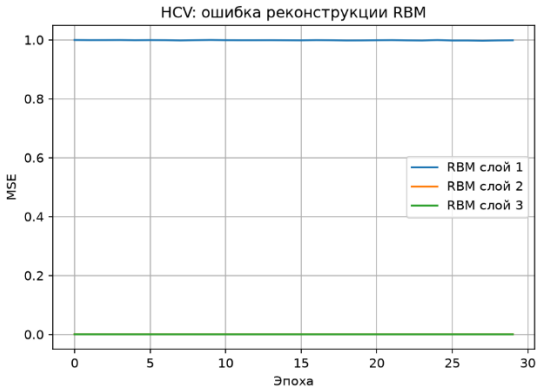


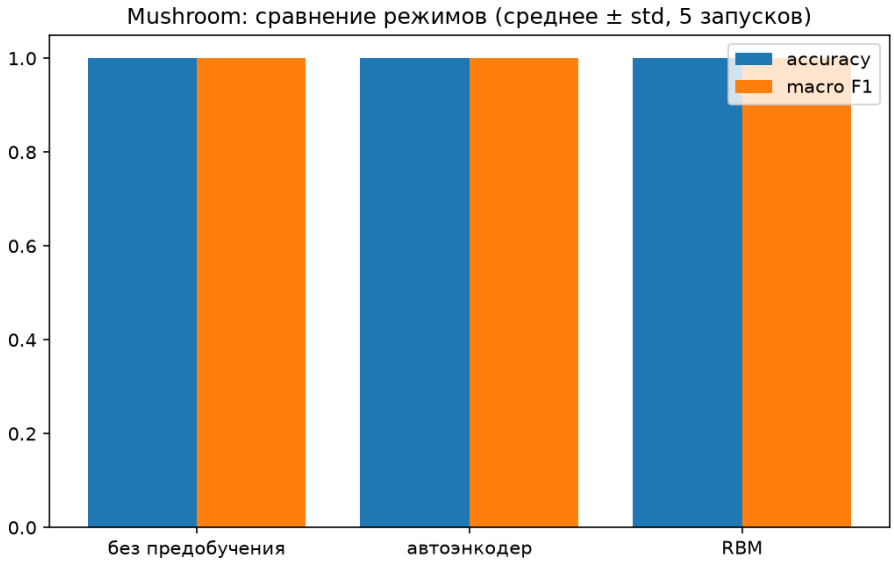

HCV: без предобучения
Истина
Предсказано
22 30 27 22
36 17 32 15
25 22 37 22
40 12 24 33

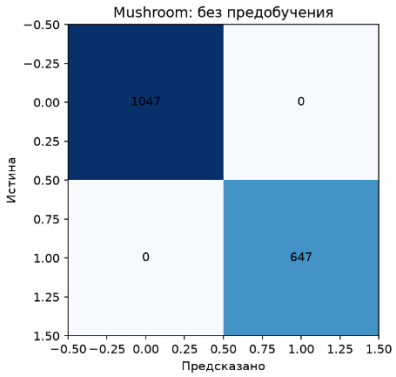

HCV: автоэнкодер
Истина
Предсказано
20 26 29 26
19 21 36 24
27 19 38 22
21 22 39 27

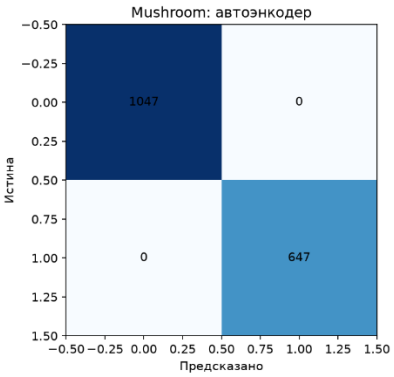

HCV: RBM
Истина
Предсказано
37 41 11 12
43 32 15 10
44 37 11 14
53 24 5 27

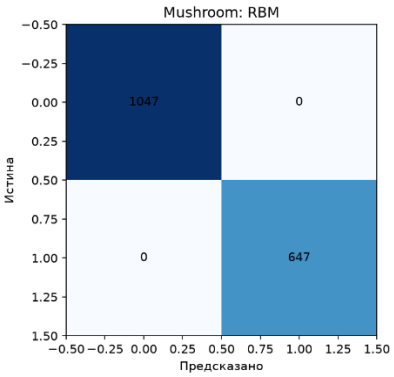

HCV: test loss при дообучении
без предобучения
автоэнкодер
RBM
Эпоха

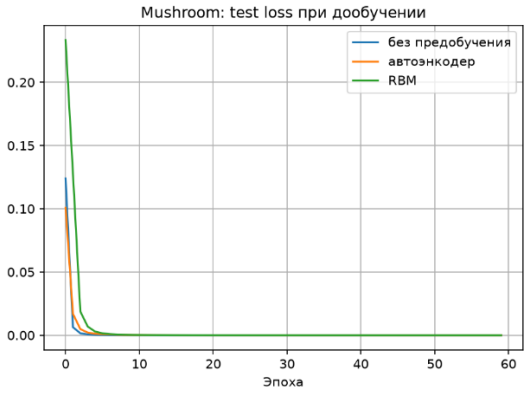

HCV: ошибка реконструкции RBM
MSE
RBM слой 1
RBM слой 2
RBM слой 3
Эпоха

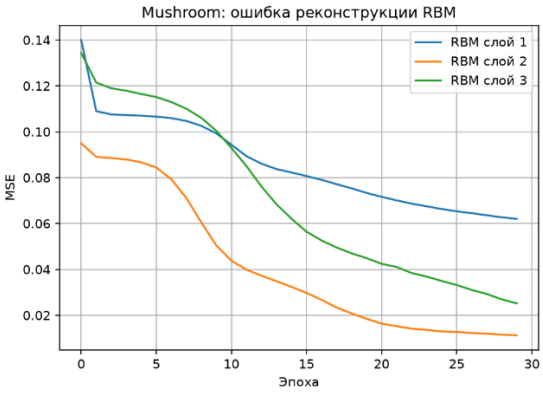

Mushroom: сравнение режимов (среднее ± std, 5 запусков)
accuracy
macro F1
без предобучения
автоэнкодер
RBM


Mushroom: без предобучения
Истина
Предсказано
1047 0
0 647


Mushroom: автоэнкодер
Истина
Предсказано
1047 0
0 647


Mushroom: RBM
Истина
Предсказано
1047 0
0 647





В работе сравнивались три режима обучения сети из четырех слоев: без предобучения, с автоэнкодерным предобучением и с предобучением стеком RBM. На датасете HCV получены accuracy 0.254, 0.258 и 0.257 и macro F1 0.251, 0.255 и 0.249 соответственно, что соответствует случайному угадыванию для четырех классов (около 0.25). Разница между режимами не превышает 0.006 и намного меньше разброса между запусками (std 0.013 до 0.026), поэтому значимой не является. Низкое качество объясняется слабой связью признаков со стадией фиброза. На датасете Mushroom все три режима дали accuracy и macro F1, равные 1.000, со стандартным отклонением 0, так как классы после one hot кодирования разделяются практически идеально. Таким образом, ни автоэнкодерное, ни RBM предобучение не улучшило качество на этих данных, и выбор способа предобучения на результат не повлиял.

Вывод: научилась осуществлять предобучение нейронных сетей с помощью RBM.